

Table of Contents

Chapter 1: Introduction to JS:	2
Section 1.1: What is JavaScript?	2
Section 1.2: Major versions of JavaScript?	2
Section 1.3: Difference Between ECMAScript, JavaScript, and Java?	3
Section 1.4: Differences Between IDE, interpreter, code editor, and compiler?	4
Section 1.5: Normal Language VS OOPS VS DSA?	5
Section 1.6: What is Syntax?	5
Section 1.7: What is an Extension?	6
Section 1.8: What is Node.js?	6
Section 1.9: What can you do with JavaScript?	7
Chapter 2: Getting started to JS:	9
Section 2.1: Window and Console?	9
Section 2.2: How to print Data in JavaScript?	10
Section 2.3: Comments & Documentation?	11
Section 2.4: Variables?	12
Section 2.4: Data Types?	14
Section 2.4: Dynamic Typing & Static Typing?	16
Section 2.4: Data Type Conversion?	17
Section 2.4: Stack and Heap Memory?	18
Section 2.4: Concatenation and Template Literals?	19
Section 2.4: String?	19
Section 2.4: Operators?	23

Chapter 1: Introduction to JS

Section 1.1: What is JavaScript?

- ✎ Imagine websites as static pictures. JavaScript brings them to life! It's like a secret code that turns boring pages into interactive experiences.
- ✎ Syntax is the rule of a programming language, which tells how a program is constructed.
- ✎ Think of clicking a button that changes color, or seeing pictures slide around. That's JavaScript in action! It's the magic behind all the cool features you see online, from games to fancy animations.
- ✎ People who make websites, called developers, use JavaScript to build all this fun stuff. It's like a superpower for the internet!
- ✎ Learning JavaScript might seem hard, but it can be just as exciting as riding a bike for the first time. It opens up a whole new world where you can create anything you imagine online!
- ✎ The father of JavaScript is Brendan Eich.
- ✎ JavaScript case sensitive: a is not equals to A.
- ✎ JavaScript, initially named Mocha and later renamed to LiveScript, was created by Brendan Eich in just 10 days in 1995 while he was working at Netscape Communications Corporation. It was first released in December 1995 as part of Netscape Navigator 2.0. The language was designed to provide interactive and dynamic functionality to web pages. In 1996, JavaScript was standardized under the name ECMAScript, with the first edition of the ECMAScript standard published. Over the years, JavaScript has evolved significantly, becoming one of the most widely used programming languages for web development, powering both client-side and server-side applications. Today, JavaScript is supported by all major web browsers and has a vast ecosystem of libraries, frameworks, and tools.

Section 1.2: Major versions of JavaScript?

Here are the major versions of JavaScript:

1. JavaScript 1.0 (Netscape Navigator 2.0, 1995)
2. JavaScript 1.1 (Netscape Navigator 3.0, 1996)
3. JavaScript 1.2 (Netscape Navigator 4.0, 1997)
4. JavaScript 1.3 (Netscape Navigator 4.06, 1997)
5. JavaScript 1.4 (Netscape Communicator 4.7, 1998)
6. JavaScript 1.5 (Netscape Communicator 4.8, 1999)
7. ECMAScript 3 (1999)
8. JavaScript 1.6 (Firefox 1.5, 2005)
9. JavaScript 1.7 (Firefox 2.0, 2006)
10. ECMAScript 4 (Abandoned)
11. ECMAScript 5 (2009)

12. ECMAScript 5.1 (2011)
13. ECMAScript 6 (ES2015, 2015)
14. ECMAScript 7 (ES2016, 2016)
15. ECMAScript 8 (ES2017, 2017)
16. ECMAScript 9 (ES2018, 2018)
17. ECMAScript 10 (ES2019, 2019)
18. ECMAScript 11 (ES2020, 2020)
19. ECMAScript 12 (ES2021, 2021)

These are the **major versions** and **releases** of the JavaScript language, along with the corresponding ECMAScript specifications.

Section 1.3: Difference Between ECMAScript, JavaScript, and Java?

The difference between **ECMAScript**, **JavaScript**, and **Java** in easy words:

1. ECMAScript:

-  ECMAScript is a scripting language specification created to standardize JavaScript.
-  It defines the syntax, semantics, and features that a scripting language like JavaScript should support.
-  ECMAScript specifications are regularly updated to introduce new features and improvements to the language.

2. JavaScript:

-  JavaScript is a programming language that implements the ECMAScript specification.
-  It is the most common implementation of ECMAScript and is used primarily for web development.
-  JavaScript allows developers to create interactive and dynamic web pages, handle browser events, and manipulate webpage content.

3. Java:

-  Java is a completely different programming language that is not related to ECMAScript or JavaScript.
-  It is a high-level, object-oriented programming language developed by Sun Microsystems (now owned by Oracle).
-  Java is commonly used for building server-side applications, mobile apps (Android), desktop applications, and large-scale enterprise systems.

In **summary**, ECMAScript is a specification for scripting languages, JavaScript is an implementation of ECMAScript used primarily for web development, and Java is a separate programming language used for various types of software development.

Section 1.4: Differences Between IDE, interpreter, code editor, and compiler?

The differences between **IDE**, **interpreter**, **code editor**, and **compiler**:

1. IDE (Integrated Development Environment):

- ✎ An IDE is a software application that provides comprehensive tools and features for software development.
- ✎ It typically includes a code editor, debugger, compiler or interpreter, build automation tools, and other features to streamline the development process.
- ✎ Examples include Visual Studio, IntelliJ IDEA, and Eclipse.

2. Interpreter:

- ✎ An interpreter is a program that directly executes code written in a high-level programming language without the need for compilation.
- ✎ It reads the source code line by line, translates it into machine code or intermediate code, and executes it immediately.
- ✎ Interpreters are commonly used for scripting languages like Python, JavaScript, and Ruby.

3. Code Editor:

- ✎ A code editor is a simple software tool used for writing and editing source code.
- ✎ It provides features like syntax highlighting, code formatting, auto-completion, and basic debugging capabilities.
- ✎ Code editors are lightweight and focused on editing code, unlike IDEs which offer a broader set of development tools.
- ✎ Examples include Sublime Text, Visual Studio Code, and Atom.

4. Compiler:

- ✎ A compiler is a program that translates source code written in a high-level programming language into machine code or bytecode.
- ✎ It processes the entire source code file or files, translates them into executable machine code, and produces an executable file.
- ✎ Compilers perform static analysis of the code, detecting errors and optimizing the code for performance.
- ✎ Examples include GCC (GNU Compiler Collection) for C/C++, Java Compiler for Java, and Swift Compiler for Swift.

In **summary**, **IDEs** provide a comprehensive development environment with tools for coding, debugging, and building applications. **Interpreters** execute code directly without compilation. **Code editors** are lightweight tools for editing source code. **Compilers** translate source code into machine code or bytecode for execution. Each serves a different purpose in the software development process.

Section 1.5: Normal Language VS OOPS VS DSA?

The differences between **normal language**, object-oriented programming (**OOP**), and data structures and algorithms (**DSA**):

1. Normal Language:

- ✎ Normal languages, such as English, Spanish, or French, are used for communication between humans.
- ✎ They follow grammar rules, syntax, and semantics specific to each language.
- ✎ Normal languages are not designed for programming or solving computational problems.

2. Object-Oriented Programming (OOP):

- ✎ Object-oriented programming is a programming paradigm based on the concept of "objects," which can contain data (attributes) and code (methods).
- ✎ It focuses on organizing code into reusable objects, allowing for modular and scalable software development.
- ✎ OOP principles include encapsulation, inheritance, and polymorphism, which help in building complex software systems.
- ✎ Examples of OOP languages include Java, C++, Python, and Ruby.

3. Data Structures and Algorithms (DSA):

- ✎ Data structures are ways of organizing and storing data in a computer's memory, while algorithms are step-by-step procedures or formulas for solving specific problems.
- ✎ DSA focuses on understanding how to efficiently store and manipulate data and how to design and analyze algorithms to solve computational problems.
- ✎ Common data structures include arrays, linked lists, stacks, queues, trees, and graphs, while algorithms include sorting, searching, graph traversal, and dynamic programming.
- ✎ DSA is essential for writing efficient and scalable software and is a fundamental topic in computer science and software engineering.

In **summary**, **normal languages** are used for human communication, while **object-oriented programming** focuses on software development using objects and classes. **Data structures and algorithms** deal with organizing and processing data efficiently to solve computational problems. Each serves a different purpose in the field of computing and software engineering.

Section 1.6: What is Syntax?

Imagine JavaScript as a language with its **own grammar** and **way of writing things**. This is **called syntax**, and it's like the **instructions** for the computer to understand what you want it to do.

Here's a sneak peek at how JavaScript looks:

- ✎ `// Those lines with two slashes are comments, like notes you write for yourself to explain the code.`
- ✎ `var message = "Hello, World!";` This line creates a special box named "message" and puts the text "Hello, World!" inside.

 `console.log(message);` This line tells the computer to take whatever is in the "message" box and show it somewhere (like a hidden notepad for the computer).

See how everything has a specific way of being written? That's JavaScript syntax at work! With semicolons at the end and parentheses around functions, it all comes together for the computer to understand.

Section 1.7: What is an Extension?

-  Imagine you have a box full of tools, but some are hidden because the box doesn't have a label. A ".js" file extension is like a label for a box of special tools for websites.
-  Just like ".mp3" tells you there's music inside and ".txt" means it's just text, ".js" lets the computer know it's a box full of JavaScript code. This code is what makes websites go from flat pictures to interactive experiences!
-  Think of buttons you can click, animations that move, or even games you can play right on a web page. Those are all thanks to JavaScript code stored in ".js" files.
-  Anyone can create these files using a simple text editor like Notepad, or a fancy tool like Visual Studio Code. Then, you just link this box of tools (the ".js" file) to your website using a special code. This way, when someone visits your site, the computer knows to use those tools to make things fun and interactive!
-  In short, ".js" files are like secret toolboxes that bring websites to life, making the web a more exciting place for everyone.

Section 1.8: What is Node.js?

Node.js is a **special tool** that helps people write code for the web. It's like a **magic box** that lets developers create all sorts of cool things for websites and web applications. But instead of using traditional programming languages like Java or Python, Node.js uses JavaScript, which is the language that makes web pages interactive.

a. How Does Node.js Work?

Imagine you're cooking in a kitchen. You have different ingredients (like JavaScript code) that you want to mix together to make a delicious meal (a web application). Node.js acts like your chef, helping you combine these ingredients in just the right way to create something amazing.

b. Key Features of Node.js

1. **JavaScript Everywhere:** With Node.js, developers can use JavaScript not only for the front-end (what users see in their web browsers) but also for the back-end (the behind-the-scenes magic that powers web applications).

2. **Fast and Scalable:** Node.js is known for its speed and efficiency, making it perfect for building real-time applications like chat apps, online games, or streaming platforms.

3. **Vast Ecosystem:** Node.js has a huge community of developers who create and share libraries, tools, and frameworks to make coding faster and easier. This means developers can build complex web applications without starting from scratch.

4. **Event-Driven Architecture:** Node.js uses a special event-driven architecture that allows it to handle multiple tasks simultaneously without getting bogged down. This makes it ideal for building highly responsive and interactive web applications.

c. Why is Node.js Important?

Node.js has revolutionized web development by making it easier for developers to build fast, scalable, and real-time applications using JavaScript. It has become a popular choice for companies big and small, from startups to tech giants like Netflix, PayPal, and LinkedIn.

In summary, Node.js is like a supercharged toolbox for web developers, empowering them to create cutting-edge web applications with ease. Whether you're building a simple website or a complex web platform, Node.js has everything you need to bring your ideas to life on the internet!

Install Node.js from: (<https://nodejs.org/en/download/current>)

Subscribe on YouTube: @NaseebCodeStudio

Section 1.9: What can you do with JavaScript?

JavaScript is used extensively in the modern world for a wide range of applications and technologies. Here are some of the things that can be created and are created using JavaScript:

1. Web Development:

-  Building dynamic and interactive websites and web applications.
-  Frontend development using frameworks like React, Angular, and Vue.js.
-  Backend development using Node.js.

2. Mobile App Development:

-  Developing cross-platform mobile apps using frameworks like React Native and Ionic.

3. Desktop App Development:

-  Creating desktop applications using frameworks like Electron.

4. Game Development:

-  Developing games using libraries and engines like Phaser.js, Three.js, and Unity (with JavaScript support).

5. Server-side Scripting:

-  Writing server-side scripts for handling server logic, file operations, database interactions, etc., using Node.js.

6. IoT (Internet of Things):

- ✎ Programming IoT devices and hardware using JavaScript libraries like Johnny-Five and platforms like Tessel.

7. Data Visualization:

- ✎ Creating interactive charts, graphs, and data visualizations using libraries like D3.js and Chart.js.

8. Machine Learning and AI:

- ✎ Building machine learning models and AI applications using TensorFlow.js and Brain.js.

9. Browser Extensions:

- ✎ Developing browser extensions/add-ons to extend the functionality of web browsers.

10. Automation:

- ✎ Automating repetitive tasks, browser automation, and testing using tools like Puppeteer and Selenium WebDriver.

11. Chatbots and Virtual Assistants:

- ✎ Building chatbots and virtual assistants for websites and messaging platforms using frameworks like Botpress and Dialogflow.

12. WebAssembly (Wasm):

- ✎ Utilizing JavaScript to work with WebAssembly for high-performance computation and running non-JavaScript code in the browser.

JavaScript's **versatility** and **extensive ecosystem** make it a powerful tool for creating a wide range of applications and technologies, contributing significantly to the modern digital landscape.

Chapter 2: Getting started to JS

Section 2.1: Window and Console?

Before moving ahead, let's understand where the console is located.

What is Window?

In JavaScript, the `window` object represents the browser window or tab that contains the JavaScript code. It is a **global object**, which means it is accessible from anywhere in your JavaScript code.

We can write these methods directly without using window keyword.

Here are some important and commonly used methods and properties of the `window` object:

1. **window.alert()**: Displays an alert dialog box with a message and an OK button.

Code:

```
window.alert("Hello, world!");
```

2. **window.prompt()**: Displays a prompt dialog box with a message, an input field for the user to enter text, and OK and Cancel buttons.

Code:

```
let userInput = window.prompt("Please enter your name:");
```

3. **window.confirm()**: Displays a confirmation dialog box with a message, OK, and Cancel buttons. Returns true if the user clicks OK and false if they click Cancel.

Code:

```
let userConfirmation = window.confirm("Are you sure you want to proceed?");
```

4. **window.open()**: Opens a new browser window or tab with the specified URL.

Code:

```
window.open("https://www.example.com", "_blank");
```

5. **window.close()**: Closes the current browser window.

Code:

```
window.close();
```

6. **window.location**: Provides information about the current URL and allows you to navigate to a new URL.

Code:

```
console.log(window.location.href); // Current URL
```

```
window.location.href = "https://www.example.com"; // Redirect to a new URL
```

7. **window.print()**: prints the complete document;

Code:

```
window.print();
```

These are just a few examples of the **methods** and **properties** provided by the `window` object in JavaScript. The `window` object also provides access to other browser-related functionality, such as working with cookies, handling events, and interacting with the browser's history and navigation.

What is console?

In JavaScript, the ``console`` object is an important tool for developers that provides methods for **logging** information to the browser's console. It is primarily used for **debugging** and **troubleshooting** purposes. The ``console`` object is available globally in the browser environment, which means you can access it from any part of your JavaScript code. Right click in a browser → inspect → navigate to the console tab.

Some commonly used methods of the ``console`` object include:

1. **`console.log()`**: Logs a message to the console. This method is commonly used to output values, variables, or debugging information.

Code:

```
console.log("Hello, world!");
```

2. **`console.error()`**: Logs an error message to the console. Errors logged with this method are typically displayed with a distinctive style in the console, making them easy to identify.

Code:

```
console.error("An error occurred!");
```

3. **`console.warn()`**: Logs a warning message to the console. Warnings logged with this method are typically displayed with a distinctive style, similar to errors.

Code:

```
console.warn("This is a warning!");
```

4. **`console.info()`**: Logs an informational message to the console. Informational messages logged with this method are typically displayed with a different style in the console, making them easy to distinguish from regular logs.

Code:

```
console.info("This is an informational message.");
```

5. **`console.table()`**: Displays tabular data in a table format in the console. This method is useful for visualizing arrays or objects with multiple properties.

Code:

```
const data = [{ name: "Naseeb_Khan", age: 22 }, { name: "Alice", age: 25 }];  
  
console.table(data);
```

The ``console`` object is a powerful tool for **debugging** and **understanding** the behavior of your JavaScript code. It allows you to **log messages, errors, warnings**, and other information directly to the browser's console, making it easier to identify and fix issues in your code.

Section 2.2: How to print Data in JavaScript?

Best practice to test code or print data by:

1. `alert()`;
2. `prompt()`;
3. `confirm()`;

4. `console.log()`
5. `console.table()`

For any errors or debugging checks, always refer to the console tab. The console shows you errors or where you have made mistakes.

Section 2.3: Comments & Documentation?

Comments in JavaScript:

In JavaScript, comments are used to add explanatory notes within the code. They are ignored by the JavaScript engine and do not affect the execution of the program. Comments are essential for improving code readability and for documenting the code for yourself and other developers.

There are two main types of comments in JavaScript:

1. **Single-line comments:** These comments begin with `//` and extend to the end of the line. They are used for brief **explanations** or **annotations** on a single line of code.

Code:

```
// This is a single-line comment
```

```
let greeting = "Hello, world!"; // This comment explains the purpose of the code
```

2. **Multi-line comments:** These comments begin with `/*` and end with `*/`. They can span multiple lines and are useful for longer **explanations** or **commenting** out blocks of code.

Code:

```
/*
```

```
This is a multi-line comment.
```

```
It can span multiple lines and is enclosed between /* and */
```

```
Multi-line comments are often used for documenting functions or describing complex algorithms.
```

```
*/
```

Comments should be used to **explain** the purpose of the code, document **important** details, provide context for future **modifications**, and **clarify** any complex or obscure sections of code. They play a **crucial role** in making the code more **understandable** and **maintainable** for both **yourself** and **other developers** who may work on the project in the future.

Best Documentation to follow for JavaScript:

There are several excellent resources available for learning JavaScript and its documentation. Here are some of the best ones:

1. **MDN Web Docs (Mozilla Developer Network):** MDN provides comprehensive documentation on all aspects of web development, including JavaScript. It covers everything from basic syntax to advanced topics, with clear explanations, examples, and reference guides.

Link:

[MDN JavaScript Guide](<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>)

2. **JavaScript.info:** This website offers in-depth tutorials and guides for learning JavaScript from scratch. It covers fundamental concepts, modern JavaScript features, and advanced topics with interactive examples and exercises.

Link:

[JavaScript.info](https://javascript.info/)

3. **W3Schools:** W3Schools provides easy-to-understand tutorials and references for web development technologies, including JavaScript. It offers a wide range of topics with simple explanations and interactive examples.

Link:

[W3Schools JavaScript Tutorial](https://www.w3schools.com/js/)

4. **ECMAScript Language Specification:** For a detailed and technical understanding of the JavaScript language, you can refer to the official ECMAScript specification. It defines the syntax, semantics, and standard library of JavaScript.

Link:

[ECMAScript Language Specification](https://www.ecma-international.org/ecma-262/
https://tc39.es/)

These resources offer a combination of **tutorials**, **guides**, and **reference** materials that cater to different learning **styles** and **levels** of expertise. Whether you're a **beginner** or an **experienced** developer, you'll find valuable information to enhance your understanding of JavaScript.

Section 2.4: Variables?

In JavaScript, variables are **containers** for **storing** data values. They are used to **hold** information that can be **referenced** and **manipulated** within a program. Variables play a **fundamental** role in programming by allowing developers to **store**, **retrieve**, and **modify** data dynamically.

Here are some key points about variables in JavaScript:

1. **Declaration:** Variables are declared using the ``var``, ``let``, or ``const`` keywords, followed by a unique identifier (variable name). For example:

Code:

```
var age; // Declaration using 'var'
let name; // Declaration using 'let'
const PI = 3.14; // Declaration and initialization using 'const'
```

2. **Initialization:** Variables can be initialized (assigned a value) at the time of declaration or later in the program. For example:

Code:

```
var age = 22; // Declaration and initialization
let name; // Declaration
name = "Naseeb Khan"; // Initialization
let a = 10, b=20, c=30; // declaration and initialization
```

3. **Naming Conventions:** Variable names can consist of letters, digits, underscores, and dollar signs. They cannot begin with a digit. It's good practice to use descriptive names that convey the purpose of the variable.

In JavaScript, there are several conventions for naming variables, each with its own style and purpose. **Here are some common naming conventions along with examples:**

- a. **Camel Case:** In camel case, the first word is all lowercase, and subsequent words are capitalized, with no spaces or punctuation between them.

Code:

```
let userName = "NaseebKhan";  
let firstName = "Naseeb";  
let lastName = "Khan";
```

- b. **Pascal Case** (Upper Camel Case): Similar to camel case, but the first letter of each word is capitalized.

Code:

```
let UserName = "NaseebKhan";  
let FirstName = "Naseeb";  
let LastName = "Khan";
```

- c. **Snake Case:** Words are separated by underscores, and all letters are lowercase.

Code:

```
let user_name = "Naseeb_Khan";  
let first_name = "Naseeb";  
let last_name = "Khan";
```

- d. **Upper Snake Case** (Screaming Snake Case): Similar to snake case, but all letters are uppercase.

Code:

```
const PI_VALUE = 3.14;  
let USER_NAME = "NaseebKhan";
```

- e. **Hungarian Notation:** Prefixes are used to indicate the type of the variable.

Code:

```
let strUserName = "NaseebKhan";  
let numAge = 22;
```

These **naming** conventions help improve code **readability** and **maintainability** by providing consistent styles for variable names. It's essential to choose a **convention** and **stick** with it throughout your codebase for clarity and consistency.

4. **Reassignment:** Variables can be reassigned (given a new value) throughout the program.

Code:

```
let count = 0;  
count = 10; // Reassignment
```

Overall, variables are essential components of JavaScript programs, enabling developers to store and manipulate data dynamically, making their code more flexible and powerful.

Section 2.4: Data Types?

Data types in programming define the kind of values that can be stored and manipulated within a program. In JavaScript, there are primitive data types, which are basic building blocks of data.

typeof keyword is used to find / print data type of data.

There are two types of Data Types:

1. Primitive data types
2. Non Primitive / Reference / Composite data types

Key Differences:

- ✎ Primitive data types store single values directly, while reference data types store references to other values or objects.
- ✎ Definition: Primitive data types are basic data types that store single values directly in memory.
- ✎ Examples: Number, String, Boolean, Undefined, Null, Symbol, BigInt.
- ✎ Primitive data types are immutable, whereas reference data types are mutable.
- ✎ Definition: Reference (or composite) data types are more complex data types that store collections of values or references to other data types.
- ✎ Examples: Object, Array, Function.
- ✎ When passed as arguments or assigned to variables, primitive data types are passed by value, while reference data types are passed by reference.

Understanding the differences between primitive and reference data types is important for writing efficient and effective code, as it affects how data is stored, manipulated, and passed around in a program.

Primitive Data Types:

- ✎ Primitive data types are often referred to as callback data types. When we copy values of primitive data types from their memory location, a copy of the value is created, not a reference to the original value. Therefore, any changes made to the copied value do not affect the original value. Instead, the changes are only committed to the copied value. This behavior ensures that the original value remains unchanged.

1. **Number:** Represents numeric values, including integers and floating-point numbers.

Example: let age = 22, a = 12.3, b = -32.4, c = -1 etc

Explanation: This is used for storing numbers, like ages, prices, or quantities.

Extra large or extra small numbers can be written with scientific (exponential) notation:

let y = 123e5; // 12300000

let z = 123e-5; // 0.00123

2. **String:** Represents textual data enclosed in single or double quotes.

Example: let name = "Naseeb Khan";

Print: console.log(name); // gets Naseeb Khan

console.log(typeof(name)); **OR** console.log(typeof name); // gets string data type

Explanation: Strings are used for storing sequences of characters, like ASCII characters (A-Z, 0-9, !@#\$, etc.), names, messages, or addresses.

3. **Boolean:** Represents a logical value of either true or false.

Example: let isStudent = true;

Explanation: Booleans are used for logical operations and representing yes/no or true/false conditions.

4. **Undefined:** Represents a variable that has been declared but not assigned a value.
Example: let x, let y = undefined;
Explanation: When a variable is declared but not initialized, its value is undefined, indicating that it has no meaningful value yet. A variable without a value is undefined and the type is also undefined. Any variable can be emptied by setting value to undefined.
5. **Null:** Represents the intentional absence of any value.
Example: let y = null;
Explanation: Null is used to explicitly indicate that a variable contains no value or that it has been intentionally set to nothing.
6. **BigInt:** Represents integers with arbitrary precision.
Example: let bigNum = 1234567890123456789012345678901234567890n;
let x = BigInt("123456789012345678901234567890");
Explanation: BigInts are used for working with very large integers that exceed the limits of regular numbers.
7. **Symbol:** To make values unique. Represents a unique and immutable value used as an identifier for object properties.
Example: let sym = Symbol("description");
Explanation: Symbols are used for creating unique identifiers, typically for object properties, to prevent naming conflicts (useful in react).

Non Primitive / Reference Data Types:

1. Array:

 **Definition:** An array is a data structure that stores a collection of elements, each identified by an index or key. Arrays are written with square brackets and values are separated by commas in array. Array indexes are 0 based, which means the first item is on a 0 place, the second is on 1 and so on. Example:

Code:

```
let emptyArr = [];  
let fruits = ["apple", "banana", "orange"];  
let position = 2;
```

Print:

```
console.log(fruits); // gets complete array  
console.log(fruits[0]); // gets apple value in array  
console.log(fruits[position]); // gets orange value in array
```

 **Explanation:** Arrays allow you to **store multiple values** in a **single variable**. Each value in the array is accessed by its index, starting from 0. Arrays can hold elements of any data type, including numbers, strings, objects, or even other arrays. They are versatile and commonly used for storing and manipulating lists of related items.

2. Object:

 **Definition:** An object is a collection of key-value pairs, where each key is a unique identifier and each value is the associated data, data can be anything. Object is written with curly braces and properties are written as name value pairs separated by comma. Example:

Code:

```
let person = {  
  name: "Naseeb Khan",  
  age: 22,
```

```
    city: "Quetta",  
    student: true  
};
```

Print:

```
console.log(person); // gets complete obj  
console.log(person.age); // gets 22 value in obj
```

- ✎ **Explanation:** Objects in JavaScript are used to represent **complex data structures**. They provide a way to **organize** and **group related data together**. Each key in an object acts as a property that holds a value. Objects can store values of **different data types**, including numbers, strings, arrays, functions, or even other objects. Objects are versatile and commonly used for **representing real-world entities**, such as people, products, or events, along with their **attributes** and **behaviors**.

3. Functions:

- ✎ Functions in JavaScript are blocks of code that are designed to perform a specific task. They allow you to encapsulate a piece of code and execute it multiple times.
- ✎ A JavaScript functions are executed when something invokes it / call it.
- ✎ Two steps to follow in a function: first, declaration/creation, and second, calling the function to obtain the result or output.

Code:

```
// Define a function named greet  
function greet() {  
    // This function simply logs a greeting message  
    console.log("Hello! Welcome to the world of JavaScript!");  
}  
// Call the greet function  
greet(); // Output: Hello! Welcome to the world of JavaScript!
```

In this example:

- ✎ We define a function named `greet` using the `function` keyword followed by the function name and parentheses `()`.
- ✎ Inside the curly braces `{}`, we write the code that we want the function to execute when called.
- ✎ We then call the `greet` function by writing its name followed by parentheses `()`. This executes the code inside the function body, which logs a greeting message to the console.

Functions are a fundamental concept in JavaScript and are used extensively for organizing code, modularization, and code reusability.

Section 2.4: Dynamic Typing & Static Typing?

1. Static programming languages:

In static programming languages, the data type of a variable is known at compile time. Once a variable is declared, its data type cannot be changed. This means that variables must be explicitly declared with their data types before they are used.

Imagine you're preparing to bake cookies. In a static language, it's like having labeled jars for each ingredient before you start baking. You know exactly what goes into each jar—flour, sugar, etc. Once you've labeled them, you can't change what's in the jars. Similarly, in static languages, when you declare a variable, you have to specify its type (like saying this jar is for flour), and once declared, you can't change its type later.

Example: In languages like C, Java, or Swift, when you declare a variable, you explicitly state its data type:

Code:

```
int num = 10; // 'num' is declared as an integer
String message = "Hello"; // 'message' is declared as a string
```

2. Dynamic programming languages

In dynamic programming languages, the data type of a variable is determined at runtime. You don't need to specify the data type when declaring a variable. Variables can hold different types of data throughout the program's execution.

Going back to the baking analogy, in dynamic languages, it's like having a single jar that can hold different ingredients at different times. You can start with flour, then put sugar in it later if you need to. Similarly, in dynamic languages, you can use a variable without specifying its type upfront, and you can change what's stored in it during the program's execution.

Example: In languages like JavaScript or Python, you don't need to specify the data type when declaring a variable:

Code:

```
let num = 10; // 'num' can hold a number
let message = "Hello"; // 'message' can hold a string
num = "World"; // Now 'num' holds a string
```

Section 2.4: Data Type Conversion?

 **Type conversion** in JavaScript involves **converting** a **value** from **one data type to another**. Here's an example demonstrating type conversion between all primitive data types:

// Converting String to Number

Code:

```
let strNum = "123";
let num = Number(strNum);
console.log(num); // Output: 123 (Number)
```

// Converting Number to String

Code:

```
let numStr = 456;
let str = String(numStr);
console.log(str); // Output: "456" (String)
```

// Converting Boolean to String

Code:

```
let bool = true;
let strBool = String(bool);
console.log(strBool); // Output: "true" (String)
```

// Converting String to Boolean

Code:

```
let strBool2 = "false";
let bool2 = Boolean(strBool2);
console.log(bool2); // Output: false (Boolean)
```

// Converting Number to Boolean

Code:

```
let numBool = 0;
let bool3 = Boolean(numBool);
console.log(bool3); // Output: false (Boolean)
```

// Converting Boolean to Number

Code:

```
let boolNum = false;
let num2 = Number(boolNum);
console.log(num2); // Output: 0 (Number)
```

In these Examples:

-  We convert a string "123" to a number using Number().
-  We convert a number 456 to a string using String().
-  We convert a boolean true to a string using String().
-  We convert a string "false" to a boolean using Boolean().
-  We convert a number 0 to a boolean using Boolean().
-  We convert a boolean false to a number using Number(). Each conversion results in the expected output for the respective data types.

Section 2.4: Stack and Heap Memory?

-  In JavaScript, memory refers to the system's ability to store and manage data during program execution. Within the memory, there are two main areas: the stack and the heap.

1. Stack:

-  The stack is a region of memory that is used for static memory allocation, which means that memory allocation is determined at compile-time.
-  It is a sequential data structure that operates in a Last In, First Out (LIFO) manner.
-  The stack is primarily used for storing primitive data types, function calls, and local variables.
-  Memory allocation and deallocation on the stack are automatic and managed by the program execution.

2. Heap:

-  The heap is a region of memory that is used for dynamic memory allocation, which means that memory allocation is determined at run-time.
-  Stores reference data type.
-  It is a more flexible and less organized memory structure compared to the stack.
-  The heap is primarily used for storing objects, arrays, and other complex data structures.
-  Memory allocation and deallocation on the heap are manual and need to be managed explicitly by the programmer.
-  Difference with an Example: Consider a scenario where we have a JavaScript function that creates an array and a local variable.

Code:

```
function createArray() { let localVariable = 10; let myArray = [1, 2, 3, 4, 5]; }
```

In this example:

-  The localVariable is stored on the stack because it is a primitive data type (number) and its memory allocation is determined at compile-time.
-  The myArray is stored on the heap because it is an array (an object) and its memory allocation is determined at run-time.

Here's a breakdown of memory usage:

Stack:

Stores the memory address of the localVariable.

Heap:

Stores the elements of the myArray (1, 2, 3, 4, 5) along with the memory addresses of each element.

Section 2.4: Concatenation and Template Literals?

 Concatenation in JavaScript refers to the process of combining two or more strings into a single string. This is typically done using the `+` operator or the `concat()` method.

Example using the `+` operator:

Code:

```
let firstName = "Naseeb";
let lastName = "Khan";
let fullName = firstName + " " + lastName;
console.log(fullName); // Output: Naseeb Khan
```

Example using the `concat()` method:

Code:

```
let firstName = "Naseeb";
let lastName = "Khan";
let fullName = firstName.concat(" ", lastName);
console.log(fullName); // Output: Naseeb Khan
```

In both examples, the `firstName` and `lastName` strings are concatenated with a space in between to form the `fullName` string. The result is then logged to the console.

Template literals

 Template literals, introduced in ES6 (ECMAScript 2015), provide a convenient way to create strings in JavaScript. They allow for embedded expressions and multiline strings without needing concatenation.

Here's a basic example of using template literals:

Code:

```
let name = "Naseeb";
let age = 22;
```

// Using template literals

Code:

```
let message = `Hello, my name is ${name} and I am ${age} years old.`;
console.log(message); // Output: Hello, my name is Naseeb and I am 22 years old.
```

In this **example**, the `\${}` syntax allows us to embed variables directly into the string. This makes the code **cleaner and more readable** compared to traditional concatenation. Additionally, template literals support multiline strings, making them useful for formatting longer text blocks.

Section 2.4: String?

Two ways to declare a string:

1. `const firstName = "Naseeb";`
2. `const lastName = new String("Khan");`

-  In JavaScript, a string is a **sequence** of **characters** enclosed within single (`' '`) or double (`" "`) quotation marks. Strings are used to represent text data and can contain letters, numbers, symbols, and whitespace.
-  Double quotes of a string inside single quote, single quote of a string inside double quotes.
-  In JavaScript, **strings** have both **properties** and **methods**. Properties are attributes that provide **information** about the string, such as its length, while methods are functions that allow you to perform **actions** or **manipulations** on the string data.

Here are some common properties and methods of strings in JavaScript:

Properties:

1. `length`: Returns the length of the string.
2. `constructor`: Returns a reference to the constructor function that created the string.

Code:

```
// Constructor function example
function Person(name, age) {
  this.name = name;
  this.age = age;
}
```

```
// Creating an instance of the Person object
let person1 = new Person("Naseeb", 22);
```

```
// Accessing the length property of a string
let str = "Hello, World!";
let strLength = str.length;
```

```
console.log("Person:", person1); // gets the complete object
console.log("String Length:", strLength); // gets the length of this string which is 13
```

In this example:

-  We define a constructor function ``Person`` with ``name`` and ``age`` parameters.
-  We create an instance of the ``Person`` object using the ``new`` keyword.
-  We declare a string variable ``str`` and access its ``length`` property using dot notation.
-  We then log both the person object and the length of the string to the console.

Methods:

1. `charAt(index)`: Returns the character at the specified index position.
2. `toUpperCase()`: Converts the string to uppercase.
3. `toLowerCase()`: Converts the string to lowercase.
4. `indexOf(substring)`: Returns the index of the first occurrence of a specified substring.
5. `substring(start, end)`: Returns a portion of the string between the specified start and end indexes.
6. `split(separator)`: Splits the string into an array of substrings based on a specified separator.
7. `trim()`: Removes whitespace from both ends of the string.
8. `concat(string2)`: Concatenates two or more strings together.
9. `startsWith(prefix)`: Checks if the string starts with the specified prefix.
10. `endsWith(suffix)`: Checks if the string ends with the specified suffix.
11. `slice(start, end)`: Returns a portion of the string between the specified start and end indexes. Similar to `substring()`, but `slice()` supports negative indexes.

12. `replace(searchValue, replaceValue)`: Replaces a specified value or pattern in the string with another value.
13. `match(regexp)`: Searches the string for a specified pattern (regular expression) and returns the found matches as an array.
14. `includes(searchString)`: Determines whether the string contains the specified substring.
15. `repeat(count)`: Returns a new string consisting of the original string repeated a specified number of times.
16. `charAt(index)`: Returns the character at the specified index position.
17. `substr(start, length)`: Returns the characters in a string beginning at the specified location through the specified number of characters.
18. `trimStart()`: Removes whitespace from the beginning of the string.
19. `trimEnd()`: Removes whitespace from the end of the string.
20. `padStart(targetLength, padString)`: Pads the current string with another string (repeated, if needed) until the resulting string reaches the specified length.
21. `padEnd(targetLength, padString)`: Pads the current string with another string (repeated, if needed) until the resulting string reaches the specified length, appending the padding to the end.

Code:

```
// Define a sample string
let sampleString = "Hello, World!";

// 1. charAt(index)
let charAtIndex = sampleString.charAt(7);
console.log("Character at index 7:", charAtIndex);

// 2. toUpperCase()
let upperCaseString = sampleString.toUpperCase();
console.log("Uppercase string:", upperCaseString);

// 3. toLowerCase()
let lowerCaseString = sampleString.toLowerCase();
console.log("Lowercase string:", lowerCaseString);

// 4. indexOf(substring)
let indexOfSubstring = sampleString.indexOf("World");
console.log("Index of 'World':", indexOfSubstring);

// 5. substring(start, end)
let substringResult = sampleString.substring(7, 12);
console.log("Substring:", substringResult);

// 6. split(separator)
let splitResult = sampleString.split(", ");
console.log("Split result:", splitResult);

// 7. trim()
let stringWithWhitespace = " Hello, World! ";
let trimmedString = stringWithWhitespace.trim();
console.log("Trimmed string:", trimmedString);

// 8. concat(string2)
let string2 = " Have a nice day!";
let concatenatedString = sampleString.concat(string2);
console.log("Concatenated string:", concatenatedString);
```

// 9. startsWith(prefix)

```
let startsWithHello = sampleString.startsWith("Hello");
console.log("Starts with 'Hello':", startsWithHello);
```

// 10. endsWith(suffix)

```
let endsWithExclamation = sampleString.endsWith("!");
console.log("Ends with '!':", endsWithExclamation);
```

// 11. slice(start, end)

```
let slicedString = sampleString.slice(7, 12);
console.log("Sliced string:", slicedString);
```

// 12. replace(searchValue, replaceValue)

```
let replacedString = sampleString.replace("World", "Universe");
console.log("Replaced string:", replacedString);
```

// Define a sample string

```
let sampleString = "Hello, JavaScript!";
```

// 13. match(regex)

```
let regex = /l+/g;
let matches = sampleString.match(regex);
console.log("Matches found:", matches);
```

// 14. includes(searchString)

```
let includesResult = sampleString.includes("Script");
console.log("Includes 'Script':", includesResult);
```

// 15. repeat(count)

```
let repeatedString = "abc".repeat(3);
console.log("Repeated string:", repeatedString);
```

// 16. charAt(index)

```
let charAtIndex = sampleString.charAt(7);
console.log("Character at index 7:", charAtIndex);
```

// 17. substr(start, length)

```
let substrResult = sampleString.substr(7, 10);
console.log("Substring:", substrResult);
```

// 18. trimStart()

```
let stringWithWhitespace = " Hello, JavaScript!";
let trimmedStartString = stringWithWhitespace.trimStart();
console.log("Trimmed start string:", trimmedStartString);
```

// 19. trimEnd()

```
let trimmedEndString = stringWithWhitespace.trimEnd();
console.log("Trimmed end string:", trimmedEndString);
```

// 20. padStart(targetLength, padString)

```
let paddedStartString = "5".padStart(4, "0");
console.log("Padded start string:", paddedStartString);
```

```
// 21. padEnd(targetLength, padString)
let paddedEndString = "10".padEnd(5, "!");
console.log("Padded end string:", paddedEndString);
```

Prototype:



stringName__proto__

In JavaScript, every object has a prototype, including strings. The prototype allows you to extend or modify the behavior of built-in objects like strings. You can add custom methods to the string prototype to enhance its functionality.

Tip:

In a website URL if you see %20 it is equal to space.

Section 2.4: Operators with Types?

TYPES OF OPERATORS:

 In JavaScript, operators are symbols that perform operations on variables and values. Here are the types of operators commonly used in JavaScript:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. String Operators
5. Logical Operators
6. Bitwise Operators
7. Ternary Operator
8. Unary Operator

1. Arithmetic operators:

 **Arithmetic operators** in JavaScript perform arithmetic calculations on numeric values.

Here's a list of arithmetic operators:

- a. Addition (+)
- b. Subtraction (-)
- c. Multiplication (*)
- d. Division (/)
- e. Remainder (%)
- f. Exponentiation (**)

Code:

// Addition

```
let result1 = 10 + 5;
console.log(result1); // Output: 15
```

// Subtraction

```
let result2 = 20 - 8;
console.log(result2); // Output: 12
```

// Multiplication

```
let result3 = 6 * 4;
console.log(result3); // Output: 24
```

// Division

```
let result4 = 100 / 5;
console.log(result4); // Output: 20
```

// Remainder

```
let result5 = 15 % 4;  
console.log(result5); // Output: 3
```

// Exponentiation

```
let result6 = 2 ** 3;  
console.log(result6); // Output: 8
```

These operators can be used with variables or directly with values.

BODMAS

 BODMAS is an acronym used to remember the order of operations in arithmetic expressions:

1. Brackets (or Parentheses)
2. Orders (exponents and roots)
3. Division and Multiplication (from left to right)
4. Addition and Subtraction (from left to right)

Here's an example applying BODMAS:

```
let result = 10 + 5 * 2 - (6 / 2) ** 2;
```

Here's how BODMAS is applied step by step:

1. **Brackets:** Resolve the expression within brackets first.
 - a. $(6 / 2) ** 2$ evaluates to $(3) ** 2$ which is 9
2. **Orders:** Evaluate any exponents.
 - b. There are no more exponents in this example.
3. **Division and Multiplication:** Perform division and multiplication from left to right.
 - c. $6 / 2$ is 3
 - d. $5 * 2$ is 10
4. **Addition and Subtraction:** Perform addition and subtraction from left to right.
 - e. $10 + 10$ is 20
 - f. $20 - 9$ is 11

So, the final result of the expression $10 + 5 * 2 - (6 / 2) ** 2$ is 11

2. Assignment Operators

 **Assignment operators:** In JavaScript, assignment operators are used to assign values to variables. They are shorthand notations for assigning values. Here are common assignment operators in JavaScript:

- a. Assign ($=$)
- b. Addition Assign ($+=$)
- c. Subtraction Assign ($-=$)
- d. Division Assign ($/=$)
- e. Multiplication Assign ($*=$)
- f. Remainder Assign ($\%=$)
- g. Exponentiation Assign ($**=$)

Code:

= (Assignment Operator): Assigns the value on the right to the variable on the left.

```
let x = 5; // x is assigned the value 5
```

+= (Addition Assignment): Adds the value on the right to the variable's current value.

```
let y = 10;  
y += 5; // Equivalent to y = y + 5; y is now 15
```

-= (Subtraction Assignment): Subtracts the value on the right from the variable's current value.

```
let z = 20;  
z -= 7; // Equivalent to z = z - 7; z is now 13
```

***= (Multiplication Assignment):** Multiplies the variable's current value by the value on the right.

```
let a = 4;  
a *= 3; // Equivalent to a = a * 3; a is now 12
```

/= (Division Assignment): Divides the variable's current value by the value on the right.

```
let b = 16;  
b /= 4; // Equivalent to b = b / 4; b is now 4
```

%= (Remainder Assignment): Assigns the remainder of the division of the variable's current value by the value on the right.

```
let c = 17;  
c %= 5; // Equivalent to c = c % 5; c is now 2
```

These assignment operators are handy for concise and efficient coding when performing arithmetic operations on variables.

3. Comparison Operators

 **Comparison operators:** Comparison operators are used in programming to compare two values and determine the relationship between them. These operators typically return a Boolean value, either `true` or `false`, based on whether the comparison is true or false.

Here are common Comparison operators in JavaScript:

- Equal (==):** Checks if two values are equal, regardless of their data types. It performs type coercion if the types are different.
- Strict Equal (===):** Checks if two values are equal and of the same data type.
- Not Equal (!=):** Checks if two values are not equal, regardless of their data types. It performs type coercion if the types are different.
- Strict Not Equal (!==):** Checks if two values are not equal or not of the same data type.
- Greater Than (>):** Checks if the left operand is greater than the right operand.
- Greater Than or Equal (>=):** Checks if the left operand is greater than or equal to the right operand.
- Less Than (<):** Checks if the left operand is less than the right operand.
- Less Than or Equal (<=):** Checks if the left operand is less than or equal to the right operand.

Here's an example demonstrating the use of these comparison operators:

Code:

```
let a = 5;  
let b = '5';  
console.log(a == b); // true - values are equal after type coercion  
console.log(a === b); // false - values are equal, but data types are different  
console.log(a != b); // false - values are equal after type coercion  
console.log(a !== b); // true - values are equal, but data types are different  
console.log(a > b); // false - '5' is not greater than 5  
console.log(a >= b); // true - '5' is equal to 5  
console.log(a < b); // false - 5 is not less than '5'  
console.log(a <= b); // true - 5 is equal to '5'
```

4. String Operators

 **String operators:** In JavaScript, string operators are used to manipulate strings, which are sequences of characters. There are several types of string operators:

Example:

- Concatenation Operator (+):** This operator is used to concatenate (join together) two or more strings into a single string.
- Assignment Operator (=):** Although primarily used for variable assignment, it can also be used to assign a string value to a variable.
- Comparison Operators (==, ===, !=, !==, >, <, >=, <=):** These operators can be used to compare strings based on their lexicographical order.
- String Methods:** While not traditional operators, JavaScript also offers a variety of methods that can be used to manipulate strings, such as `toUpperCase()`, `toLowerCase()`, `slice()`, `substring()`, `charAt()`, `indexOf()`, etc.

5. Bitwise Operators

 **Bitwise operators:** Bitwise operators in JavaScript are used to manipulate the binary representations of numbers at the bit level. They allow you to perform operations such as shifting bits left or right, performing bitwise AND, OR, XOR, and NOT operations.

Here are the types of bitwise operators in JavaScript:

- Bitwise AND (&):** Performs a bitwise AND operation on each pair of corresponding bits. It returns a 1 in each bit position where both operands have a 1.
- Bitwise OR (|):** Performs a bitwise OR operation on each pair of corresponding bits. It returns a 1 in each bit position where at least one operand has a 1.
- Bitwise XOR (^):** Performs a bitwise XOR (exclusive OR) operation on each pair of corresponding bits. It returns a 1 in each bit position where the operands have different bits.
- Bitwise NOT (~):** Performs a bitwise NOT operation on each bit, resulting in the complement of the operand.
- Left Shift (<<):** Shifts the bits of the operand to the left by a specified number of positions.
- Sign-propagating Right Shift (>>):** Shifts the bits of the operand to the right by a specified number of positions, while preserving the sign bit (the leftmost bit).
- Zero-fill Right Shift (>>>):** Shifts the bits of the operand to the right by a specified number of positions, filling the leftmost positions with zeros.

Here's a simple code example demonstrating the use of bitwise operators:

Code:

```
// Bitwise AND (&)
const resultAND = 5 & 3; // Decimal: 5 & 3 = 1
console.log(resultAND); // Output: 1

// Bitwise OR (|)
const resultOR = 5 | 3; // Decimal: 5 | 3 = 7
console.log(resultOR); // Output: 7

// Bitwise XOR (^)
const resultXOR = 5 ^ 3; // Decimal: 5 ^ 3 = 6
console.log(resultXOR); // Output: 6

// Bitwise NOT (~)
const resultNOT = ~5; // Decimal: ~5 = -6
console.log(resultNOT); // Output: -6

// Left Shift (<<)
const resultLeftShift = 5 << 1; // Decimal: 5 << 1 = 10
```

```

console.log(resultLeftShift); // Output: 10

// Sign-propagating Right Shift (>>)
const resultRightShift = 5 >> 1; // Decimal: 5 >> 1 = 2
console.log(resultRightShift); // Output: 2

// Zero-fill Right Shift (>>>)
const resultZeroFillShift = -5 >>> 1; // Decimal: -5 >>> 1 = 2147483645
console.log(resultZeroFillShift); // Output: 2147483645

```

In this example, each operation is performed on two numbers, and the comment next to each operation shows the expected result in decimal form.

6. Logical Operators

 **Logical operators:** In JavaScript, logical operators are used to perform logical operations on Boolean values. There are three types of logical operators:

Here are the types of Logical operators in JavaScript:

- Logical AND (`&&`):** Returns true if both operands are true.
- Logical OR (`||`):** Returns true if at least one of the operands is true.
- Logical NOT (`!`):** Returns the opposite of the Boolean value of the operand. If the operand is true, it returns false. If the operand is false, it returns true.

Code:

```

// Logical AND (&&) example
const x = 10;
const y = 5;
const z = 20;
const resultAnd = (x < y) && (y < z); // Evaluates to false
console.log(resultAnd); // Output: false

```

```

// Logical OR (||) example
const a = 15;
const b = 30;
const c = 25;
const resultOr = (a > b) || (b < c); // Evaluates to true
console.log(resultOr); // Output: true

```

```

// Logical NOT (!) example
const isRaining = false;
const isNotRaining = !isRaining; // Evaluates to true
console.log(isNotRaining); // Output: true

```

In this example:

-  For the logical AND operator, the expression `(x < y) && (y < z)` evaluates to false because the first condition `(x < y)` is false.
-  For the logical OR operator, the expression `(a > b) || (b < c)` evaluates to true because the second condition `(b < c)` is true.
-  For the logical NOT operator, it flips the boolean value. So, `!isRaining` evaluates to true since `isRaining` is false.

7. Ternary Operator

 **Ternary operator:** In JavaScript, the ternary operator is a concise way to write conditional statements. It takes three operands: a condition followed by a question mark (`?`), a value to return if the condition is true, and a value to return if the condition is false.

Here's the syntax:

```
condition ? expression1 : expression2;
```

There's only one type of ternary operator in JavaScript, but it can be used in various ways to achieve different outcomes. Here's a breakdown:

Basic Ternary Operator: This is the simplest form where you have a single condition.

```
let result = condition ? valueIfTrue : valueIfFalse;
```

Nested Ternary Operator: You can nest ternary operators to handle multiple conditions, but this might reduce code readability if used excessively.

```
let result = condition1 ? value1 : (condition2 ? value2 : value3);
```

Chained Ternary Operator: You can chain multiple ternary operators to handle more complex scenarios, similar to nested if-else statements.

```
let result = condition1 ? value1 : condition2 ? value2 : value3;
```

Here's an example illustrating each type:

Code:

```
// Basic Ternary Operator
let age = 20;
let message = (age >= 18) ? "You are an adult" : "You are a minor";
console.log(message); // Output: "You are an adult"

// Nested Ternary Operator
let num = 10;
let result = (num > 0) ? "Positive" : (num < 0) ? "Negative" : "Zero";
console.log(result); // Output: "Positive"

// Chained Ternary Operator
let x = 5, y = 10;
let max = (x > y) ? x : (y > x) ? y : "They are equal";
console.log(max); // Output: 10
```

In each case, the result depends on the condition being evaluated.

8. Unary Operators

 **Unary operator:** Unary operators in JavaScript are operators that operate on a single operand. They perform various operations like incrementing/decrementing a value, converting a value to a different type, or negating a value.

Here are some types of unary operators in JavaScript:

postfix and prefix increment/decrement operators.

Postfix Increment (++) and Decrement (--):

 The postfix increment (++) and decrement (--) operators are used to increment or decrement the value of a variable by 1 after the current value of the variable is used in the expression.

 For example, in the expression `a++`, the value of `a` will be used first in the expression, and then it will be incremented by 1.

- ✎ Similarly, in the expression ``a--``, the value of ``a`` will be used first in the expression, and then it will be decremented by 1.

Code:

```
var a = 5;  
var b = a++; // b will be assigned the value of a (5), then a will be incremented by 1  
console.log("a:", a); // Output: a: 6  
console.log("b:", b); // Output: b: 5
```

Prefix Increment (++) and Decrement (--):

- ✎ The prefix increment (++) and decrement (--) operators are used to increment or decrement the value of a variable by 1 before the current value of the variable is used in the expression.
- ✎ For example, in the expression ``++a``, the value of ``a`` will be incremented by 1 first, and then the updated value will be used in the expression.
- ✎ Similarly, in the expression ``--a``, the value of ``a`` will be decremented by 1 first, and then the updated value will be used in the expression.

Code:

```
var x = 10;  
var y = ++x; // x will be incremented by 1, then assigned to y  
console.log("x:", x); // Output: x: 11  
console.log("y:", y); // Output: y: 11
```

So, in summary:

- ✎ Postfix increment/decrement (``a++`` or ``a--``) uses the current value of the variable in the expression and then increments/decrements it.
- ✎ Prefix increment/decrement (``++a`` or ``--a``) increments/decrements the variable first and then uses the updated value in the expression.